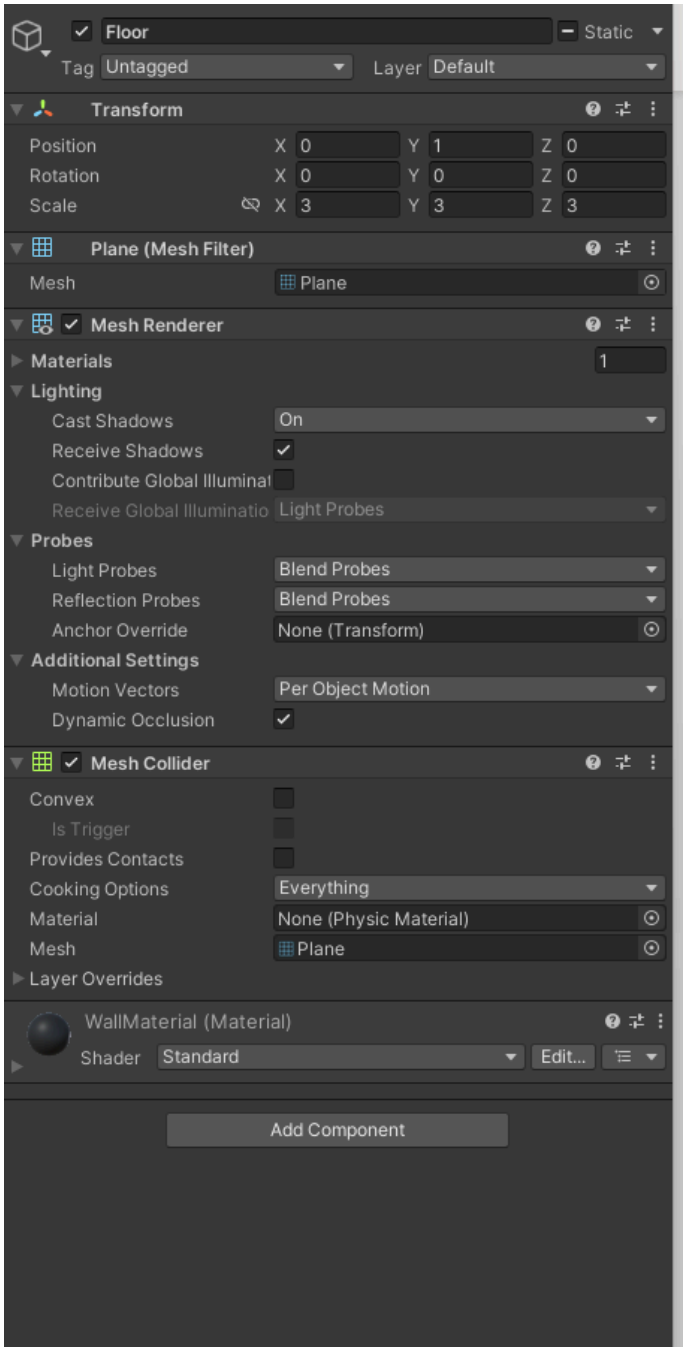
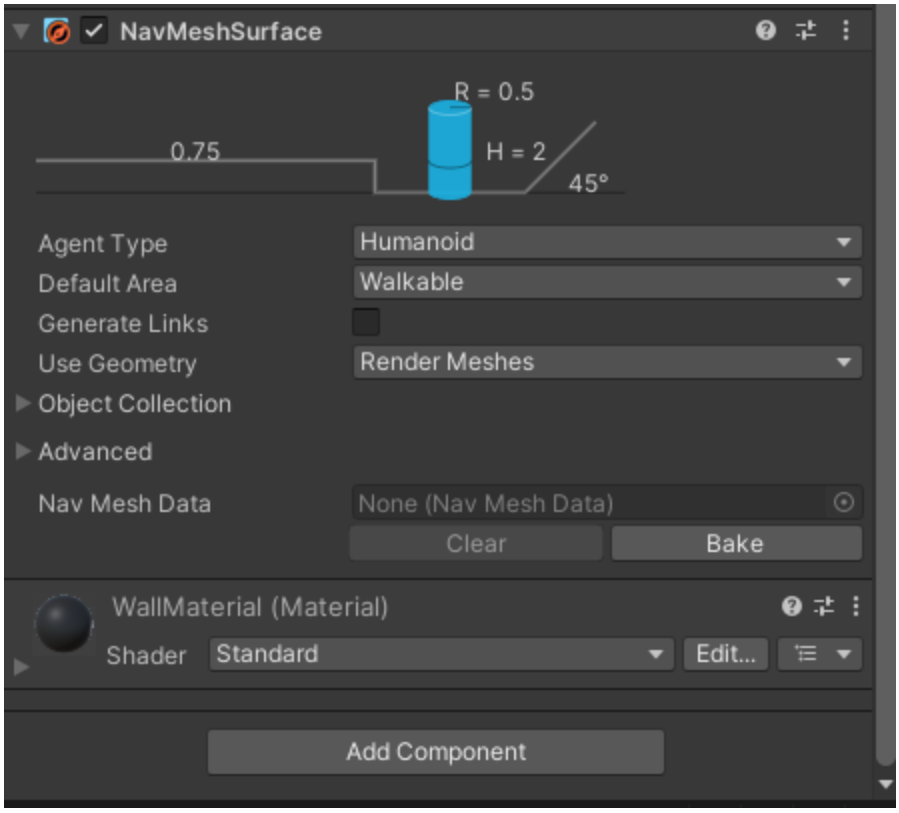
Lab 7 Navigation :

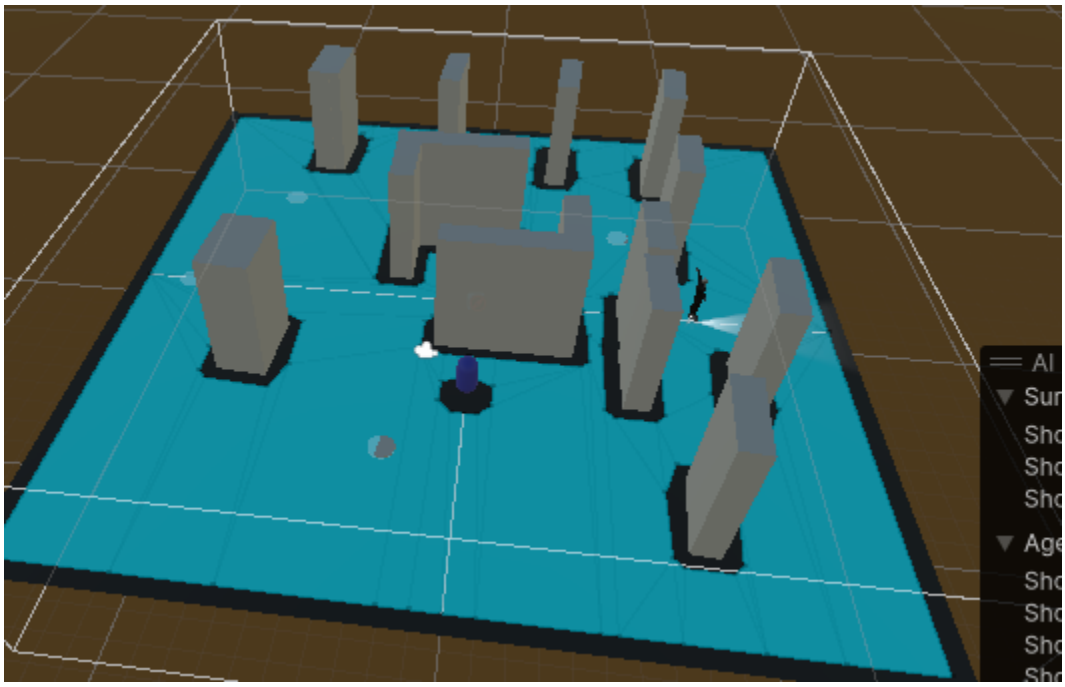
Accept the clone the github repo :https://github.com/amcnabbbaltar/420-541-Lab9

Step 1: Setup your scene

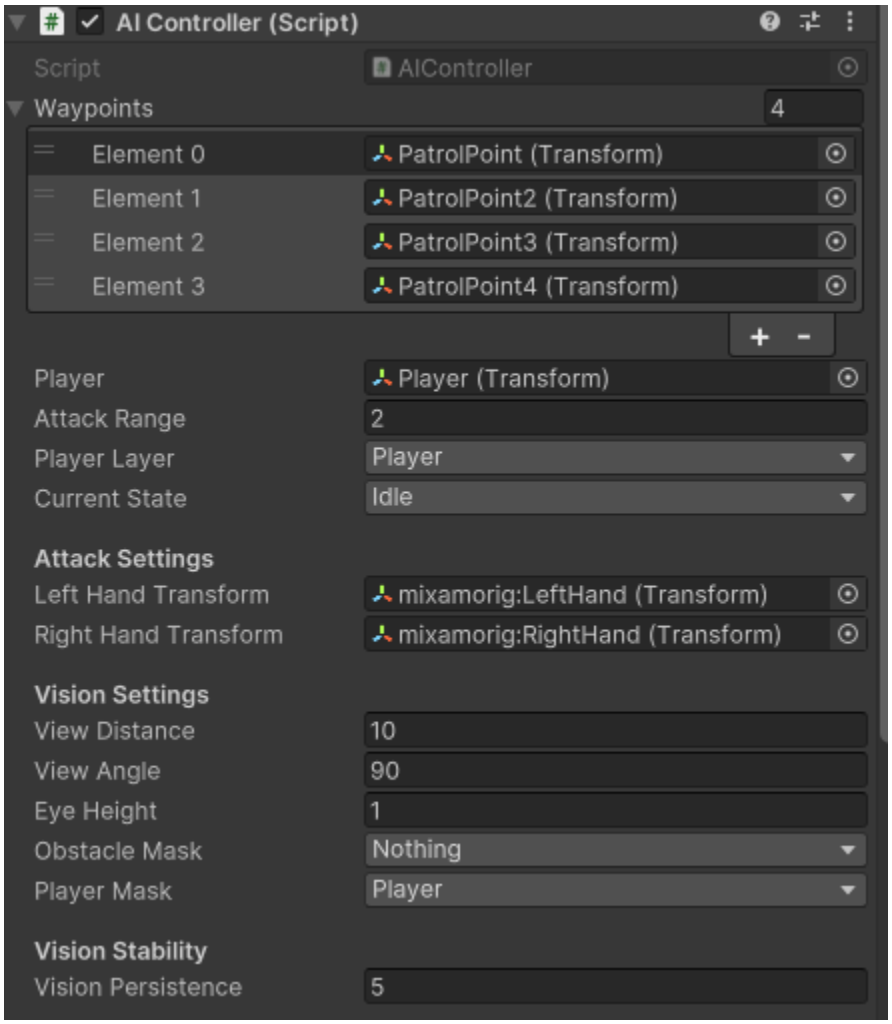
1. **In the inspector add a NavMesh Surface component to the Floor**



2. 

3. **Make sure that we ignore the player and the enemy**
4. **Then click Bake to bake the navmesh**
5. 

We will now setup the zombie AI Controller it will need to look something like this :



Now lets Create the AI State Machine Classes

3.1 Base State Class

Create an interface called IState:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public interface IState
{
    StateType Type { get; }
```

```
    void Enter();
    void Execute();
    void Exit();
}
```

3.2 Individual State Classes

For each behavior (Idle, Patrol, Chase, and Attack), create a derived class that inherits from IState:

IdleState:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class IdleState : IState
{
    private AIController aiController;
    private float idleDuration = 2f;
    private float idleTimer;

    public StateType Type => StateType.Idle;

    public IdleState(AIController aiController)
    {
        this.aiController = aiController;
    }

    public void Enter()
    {
        idleTimer = 0f;
        //aiController.Animator.SetBool("isMoving", false);
    }

    public void Execute()
    {
        idleTimer += Time.deltaTime;
        if (idleTimer >= idleDuration)
```

```
        {
            aiController.StateMachine.TransitionToState(StateType.Patrol);
        }
    }

    public void Exit()
    {
        // Cleanup if necessary
    }
}
```

PatrolState:

```
using UnityEngine;
using System.Collections;

public class PatrolState : IState
{
    private AIController aiController;
    private int currentWaypointIndex = 0;
    private bool isWaiting = false;

    public StateType Type => StateType.Patrol;

    public PatrolState(AIController aiController)
    {
        this.aiController = aiController;
    }

    public void Enter()
    {

        aiController.Agent.isStopped = false;
        MoveToNextWaypoint();
    }
```

```
    public void Execute()
    {
        if (aiController.CanSeePlayer())
        {
            aiController.StateMachine.TransitionToState(StateType.Chase);
            return;
        }

        if (!isWaiting && !aiController.Agent.pathPending &&
aiController.Agent.remainingDistance <= aiController.Agent.stoppingDistance)
        {
            aiController.StartCoroutine(WaitAndAnimate());
        }

    }

    public void Exit()
    {
        aiController.Agent.isStopped = false;
    }

    private IEnumerator WaitAndAnimate()
    {
        isWaiting = true;
        aiController.Agent.isStopped = true;

        // Play patrol point animation
        aiController.aiAnimationController.animator.SetTrigger("doScream");

        // Wait for animation duration (1.5 sec here, adjust to your animation
length)
        yield return new WaitForSeconds(5);

        aiController.Agent.isStopped = false;
        MoveToNextWaypoint();
        isWaiting = false;
    }

    private void MoveToNextWaypoint()
```

```
    {
        if (aiController.Waypoints.Length == 0)
            return;

        aiController.Agent.destination =
aiController.Waypoints[currentWaypointIndex].position;
        currentWaypointIndex = (currentWaypointIndex + 1) %
aiController.Waypoints.Length;
    }
}
```

Chase State

```
public class ChaseState : IState
{
    private AIController aiController;

    public StateType Type => StateType.Chase;

    public ChaseState(AIController aiController)
    {
        this.aiController = aiController;
    }

    public void Enter()
    {
        // No animations, so no need to set any animator parameters
    }

    public void Execute()
    {
        if (!aiController.CanSeePlayer())
        {
            aiController.StateMachine.TransitionToState(StateType.Patrol);
            return;
        }
```

```
        if (aiController.IsPlayerInAttackRange())
        {
            aiController.StateMachine.TransitionToState(StateType.Attack);
            return;
        }

        aiController.Agent.destination = aiController.Player.position;
    }

    public void Exit()
    {
        // No cleanup necessary
    }
}
```

AttackState:

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class AttackState : IState
{
    private AIController aiController;
    private float attackCooldown = 1.5f; // seconds between attacks
    private float lastAttackTime = -999f;

    public StateType Type => StateType.Attack;

    public AttackState(AIController aiController)
    {
        this.aiController = aiController;
    }
```

```
    public void Enter()
    {
        aiController.Agent.isStopped = true;
    }

    public void Execute()
    {
        // If player moves out of attack range, go back to chase
        if (Vector3.Distance(aiController.transform.position,
aiController.Player.position) > aiController.AttackRange)
        {
            aiController.StateMachine.TransitionToState(StateType.Chase);
            return;
        }

        // Limit how often the AI attacks
        if (Time.time - lastAttackTime > attackCooldown)
        {
            lastAttackTime = Time.time;
            aiController.aiAnimationController.animator.SetTrigger("doAttack");
        }
    }



    public void Exit()
    {
        aiController.Agent.isStopped = false;
    }
}
```

Step 4: Create a class called StateMachine:

Create the State Machine Class

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class StateMachine
{
    private Dictionary<StateType, IState> states = new Dictionary<StateType,
IState>();
    private IState currentState;

    public StateType GetCurrentStateType()
    {
        return currentState.Type;
    }

    public void AddState(IState state)
    {
        if (!states.ContainsKey(state.Type))
        {
            states.Add(state.Type, state);
        }
    }

    public void TransitionToState(StateType type)
    {
        currentState?.Exit();
        currentState = states[type];
        currentState.Enter();
    }

    public void Update()
    {
        currentState?.Execute();
    }
}
```

Now lets modify our AIController to add the StateMachine
Put this variable in the list of variable

```
public StateMachine StateMachine { get; private set; }
```

Then add the following functions to the script

```
    void Start()
    {
        Agent = GetComponent<NavMeshAgent>();
        aiAnimationController = GetComponent<AIAnimationController>();
        // Animator = GetComponent<Animator>(); // Commented out since we're not
using animations

        StateMachine = new StateMachine();
        StateMachine.AddState(new IdleState(this));
        StateMachine.AddState(new PatrolState(this));
        StateMachine.AddState(new ChaseState(this));
        StateMachine.AddState(new AttackState(this)); // Add the new AttackState

        StateMachine.TransitionToState(StateType.Idle);
    }

    void Update()
    {
        StateMachine.Update();
        currentState = StateMachine.GetCurrentStateType();
    }
```

Now lets create fill in the AIAnimationController. The script responsible for sending animation information to our animator from the navmesh agent.

```
using UnityEngine;

using UnityEngine.AI;

using UnityEngine.SceneManagement;
```

```
[RequireComponent(typeof(Animator))]
public class AIAnimationController : MonoBehaviour
{
    public  Animator animator { get; private set; }
    private AIController aiController;
    private NavMeshAgent agent;



    void Awake()
    {
        animator = GetComponent<Animator>();
        aiController = GetComponent<AIController>();
        agent = GetComponent<NavMeshAgent>();
    }

    void Update()
    {
        // Update animation parameters
        UpdateAnimations();
    }

    void UpdateAnimations()
```

```
    {
        float speed = agent != null ? agent.velocity.magnitude : 0f;
        animator.SetFloat("CharacterSpeed", speed);
    }

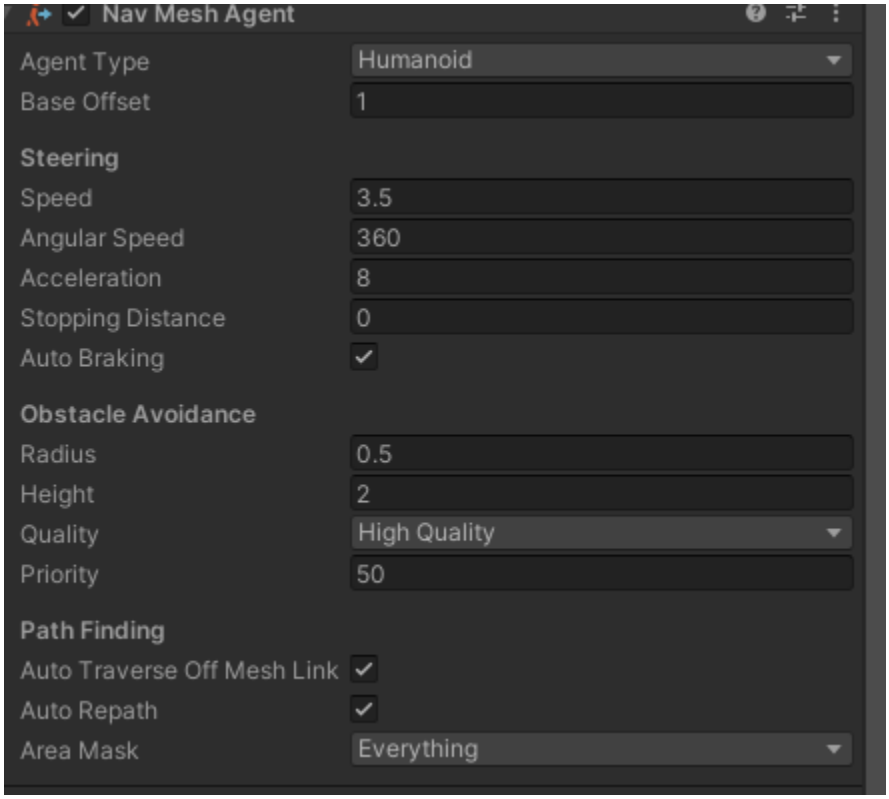
    void HitPlayer() // Attack Animation Event to check if the player his hit
    {
        GameObject objectHit;
        if (aiController.CheckHandsCollision(out objectHit, "Player"))
        {

            //SceneManager.LoadScene(0);
        }
    }

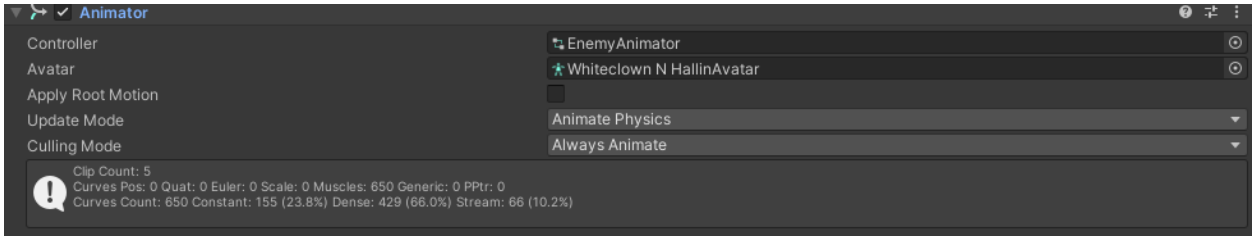
}
```

Step 8: Assign Components in Unity

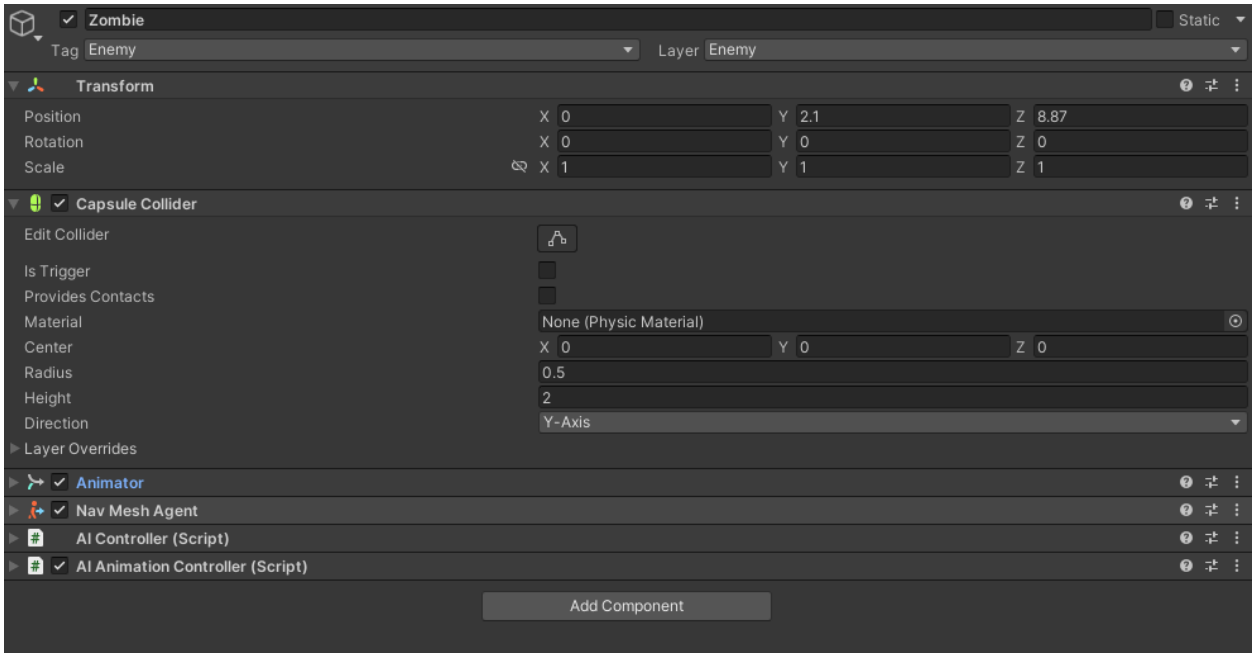
1. Assign the NavMeshAgent in the inspector.

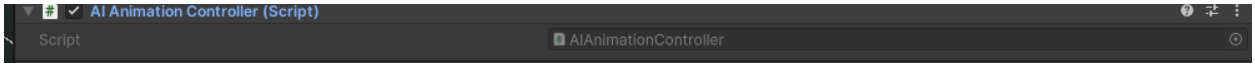


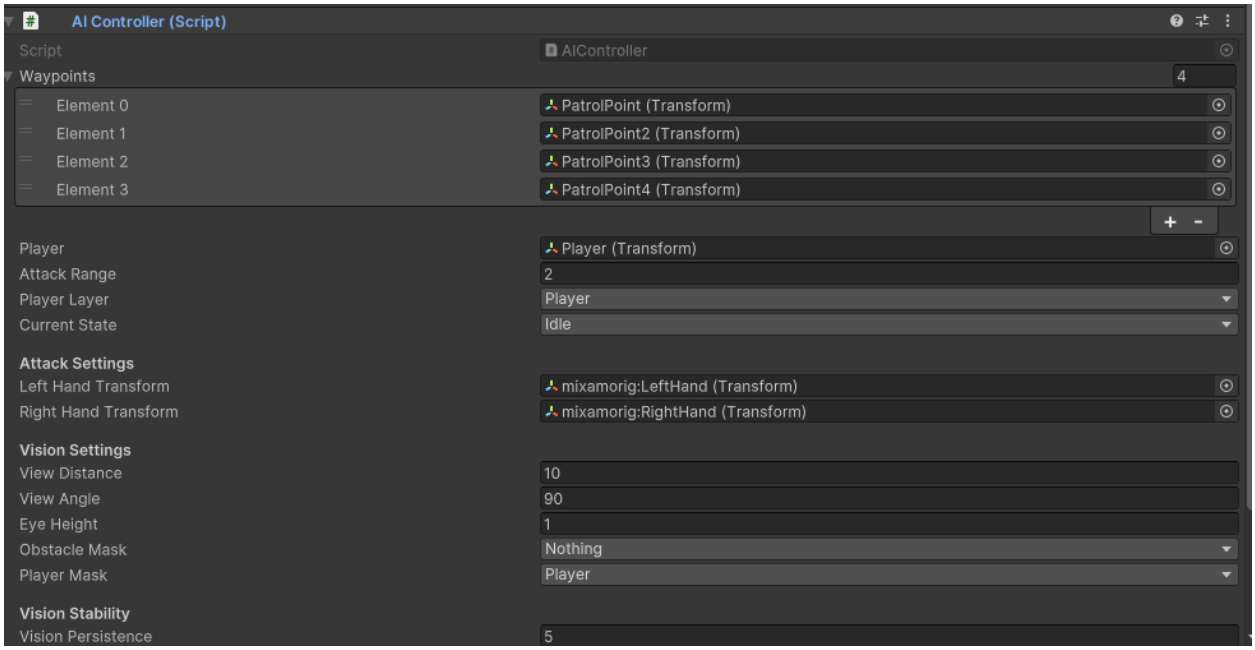
2. Make sure the NavMeshAgent base offset is set to 1.
3. Assign the AI AnimationController that we just created
4. Add an animator that has the EnemyAnimator And the WhiteClownAvatar

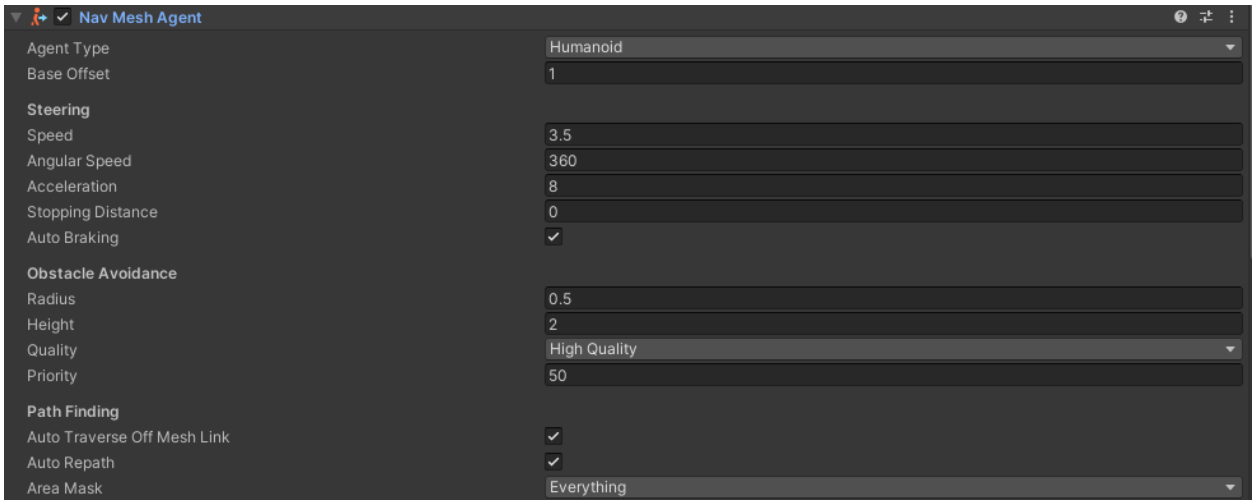
5. 

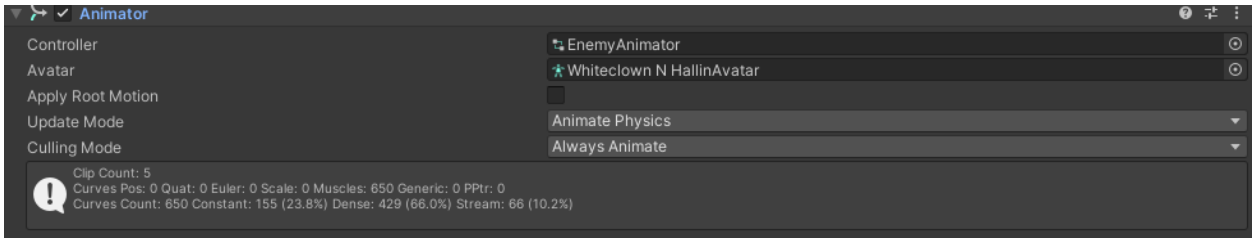
6. By the end your zombie game object should look something like this

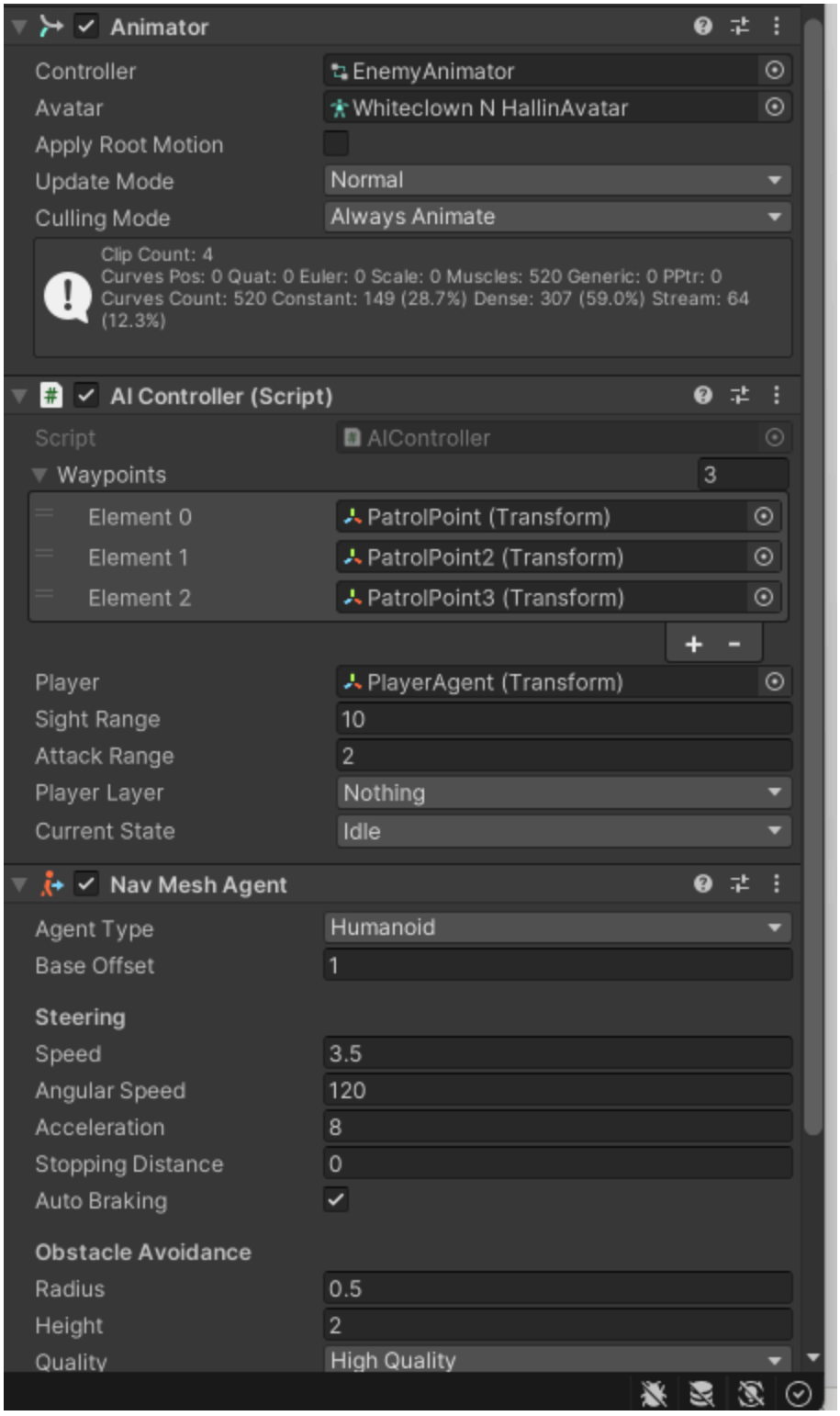
7. 

8. 

9. 
AI Controller (Script)
Script
AIController
Waypoints
4
Element 0
PatrolPoint (Transform)
Element 1
PatrolPoint2 (Transform)
Element 2
PatrolPoint3 (Transform)
Element 3
PatrolPoint4 (Transform)
Player
Player (Transform)
Attack Range
2
Player Layer
Player
Current State
Idle
Attack Settings
Left Hand Transform
mixamorig:LeftHand (Transform)
Right Hand Transform
mixamorig:RightHand (Transform)
Vision Settings
View Distance
10
View Angle
90
Eye Height
1
Obstacle Mask
Nothing
Player Mask
Player
Vision Stability
Vision Persistence
5

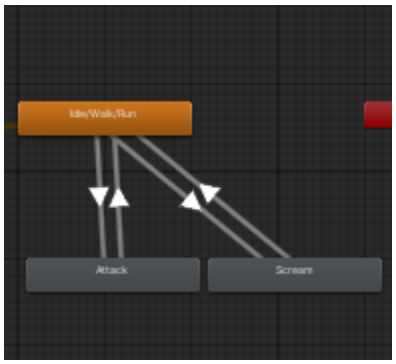
10. 
Nav Mesh Agent
Agent Type
Humanoid
Base Offset
1
Steering
Speed
3.5
Angular Speed
360
Acceleration
8
Stopping Distance
0
Auto Braking
Obstacle Avoidance
Radius
0.5
Height
2
Quality
High Quality
Priority
50
Path Finding
Auto Traverse Off Mesh Link
Auto Repath
Area Mask
Everything

11. 
Animator
Controller
EnemyAnimator
Avatar
Whiteclown N HallinAvatar
Apply Root Motion
Update Mode
Animate Physics
Culling Mode
Always Animate
Clip Count: 5
Curves Pos: 0 Quat: 0 Euler: 0 Scale: 0 Muscles: 650 Generic: 0 PPtr: 0
Curves Count: 650 Constant: 155 (23.8%) Dense: 429 (66.0%) Stream: 66 (10.2%)

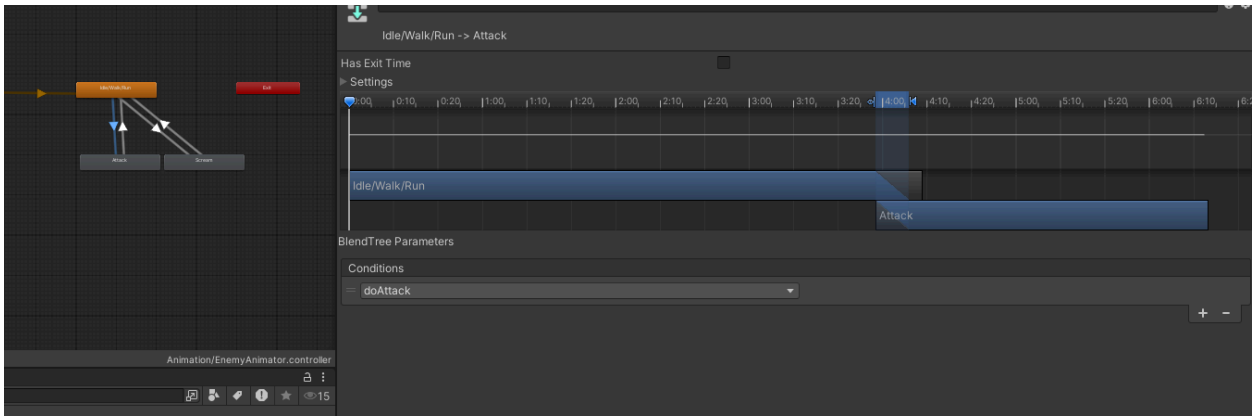


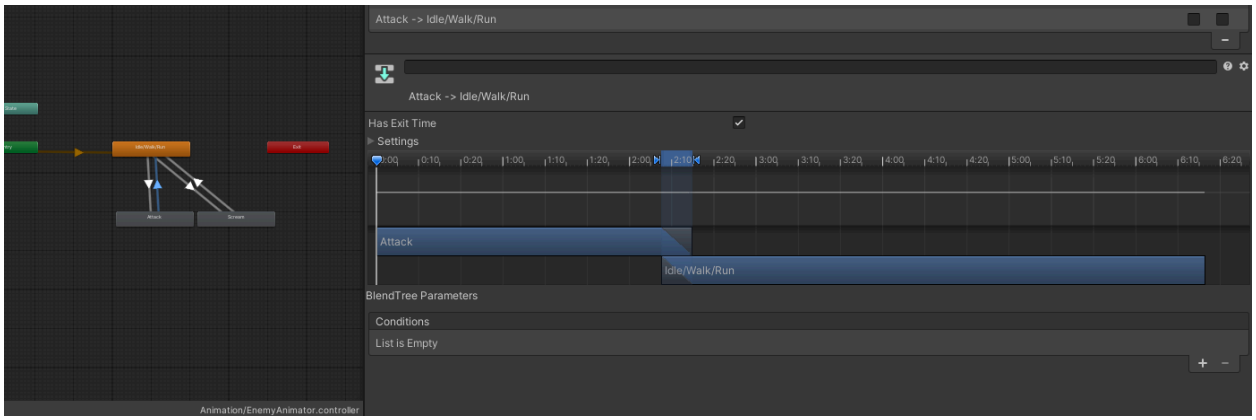
Setup the animator controller

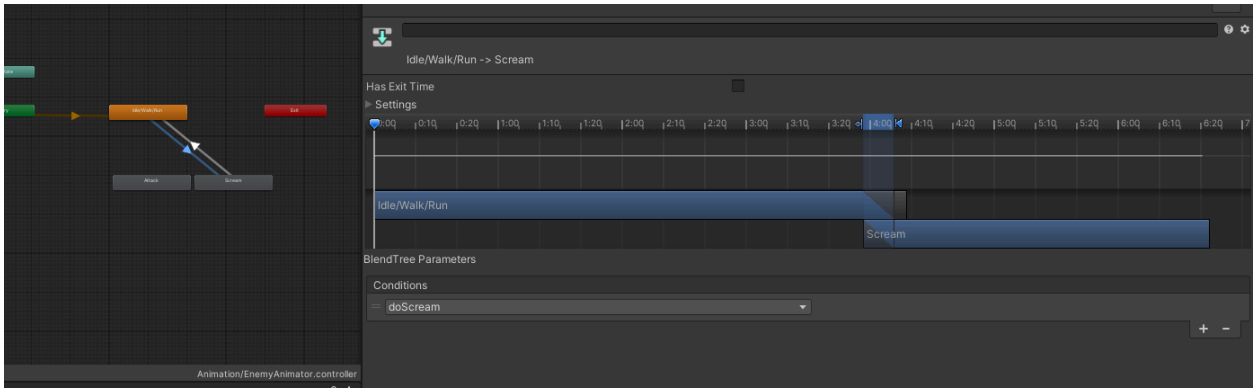
Setup the animator like this

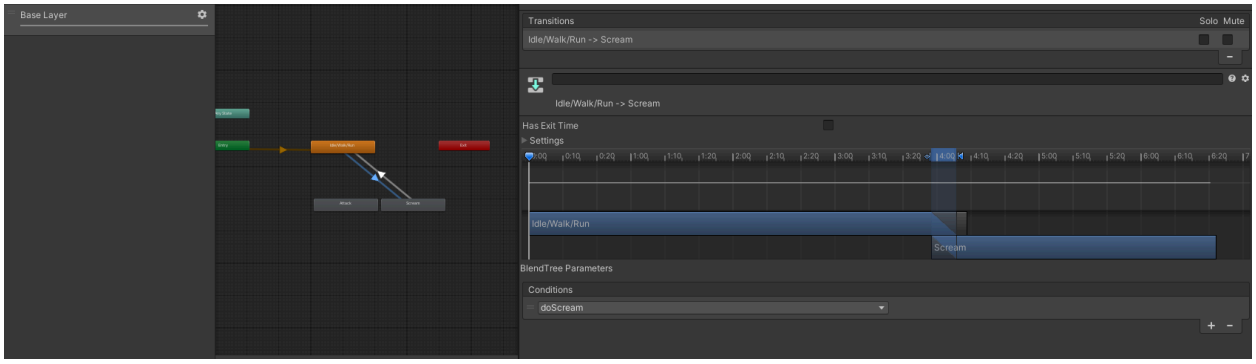


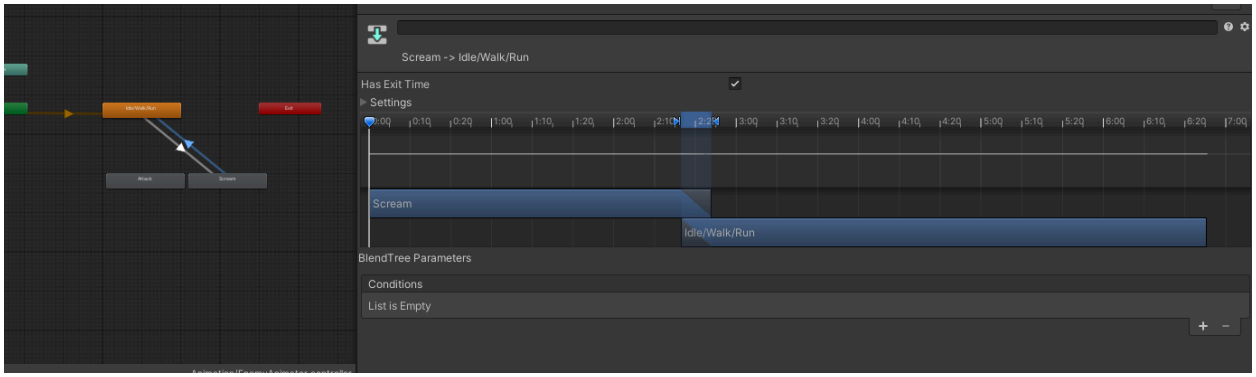
Set up the transition as follows



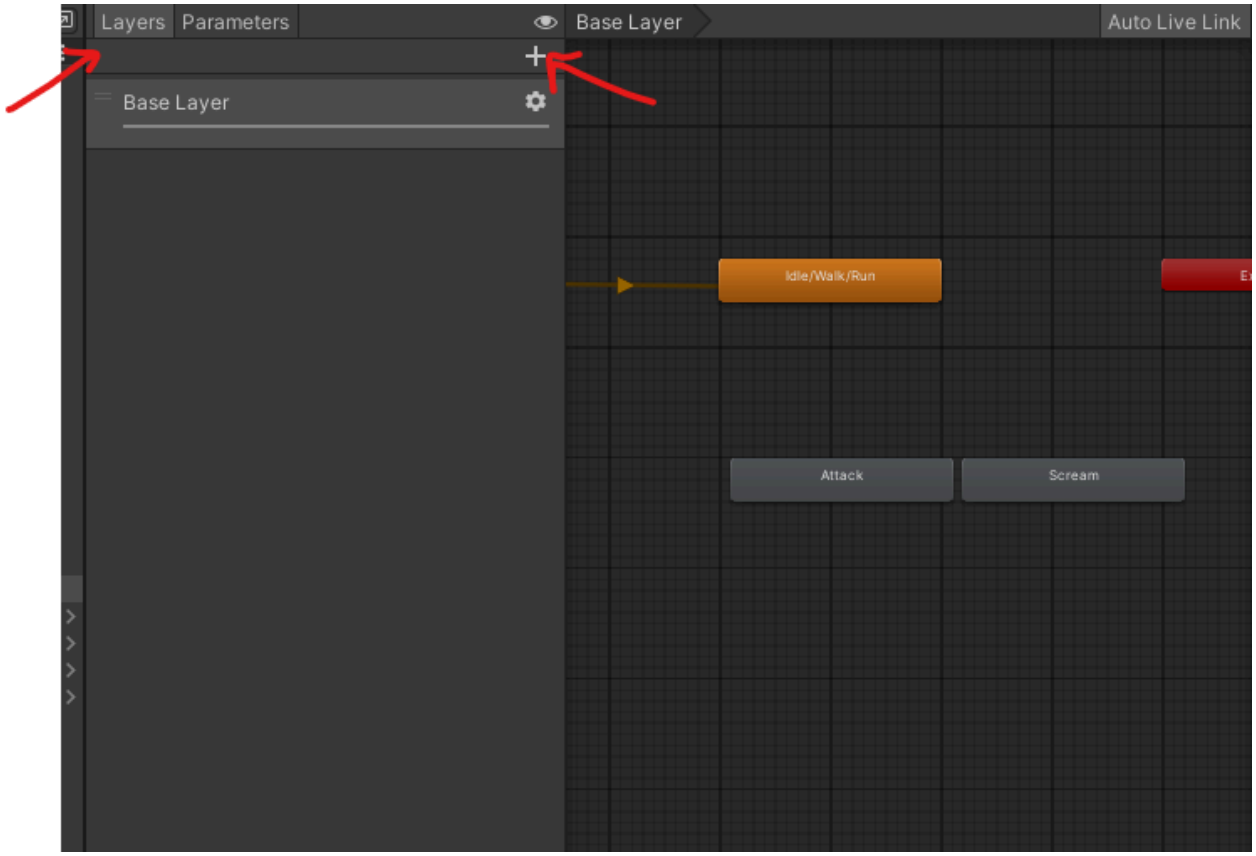




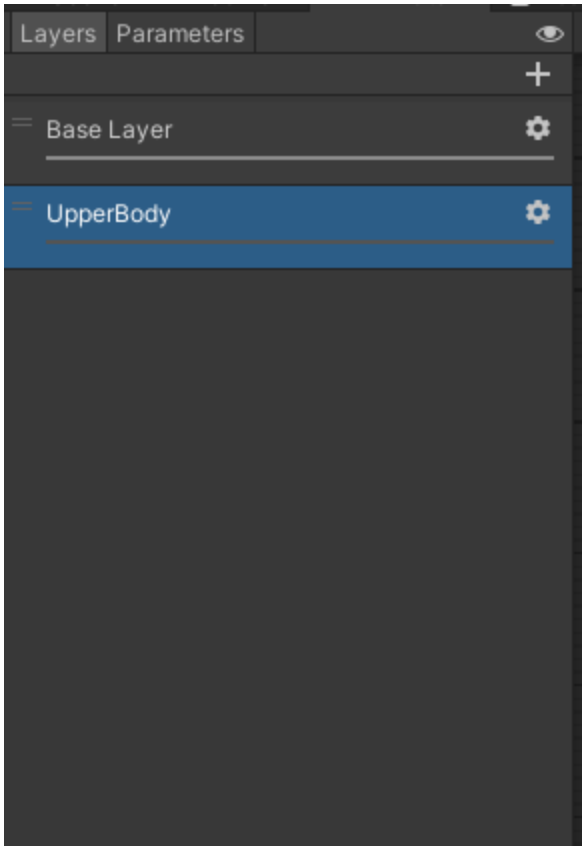




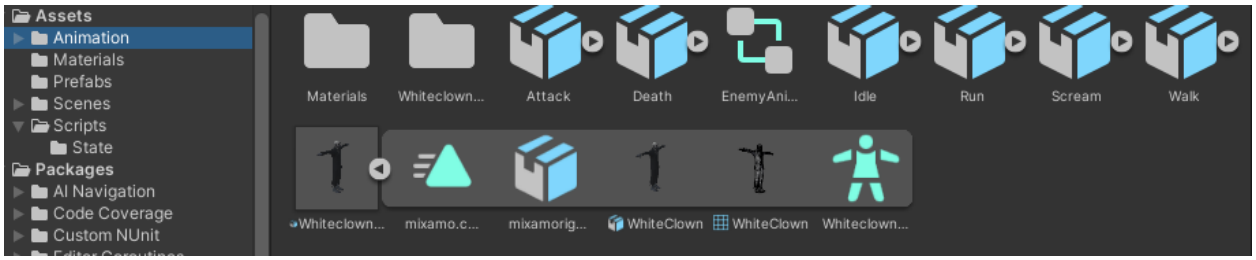
You will notice that when the AI attacks you if you back out he will keep sliding we will fix this by making the attack a layered animation.



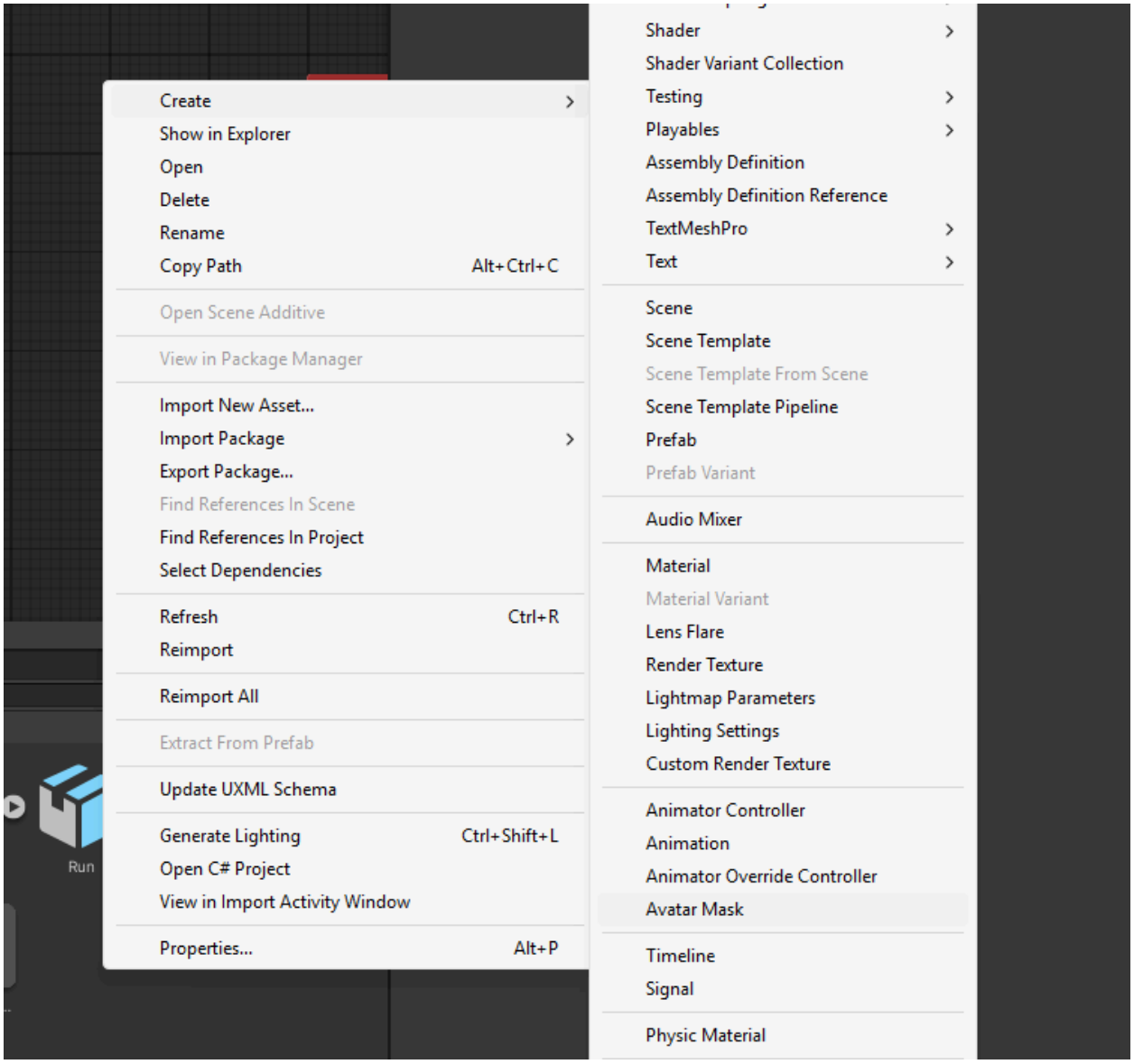
We will create a Layer that we will call UpperBody



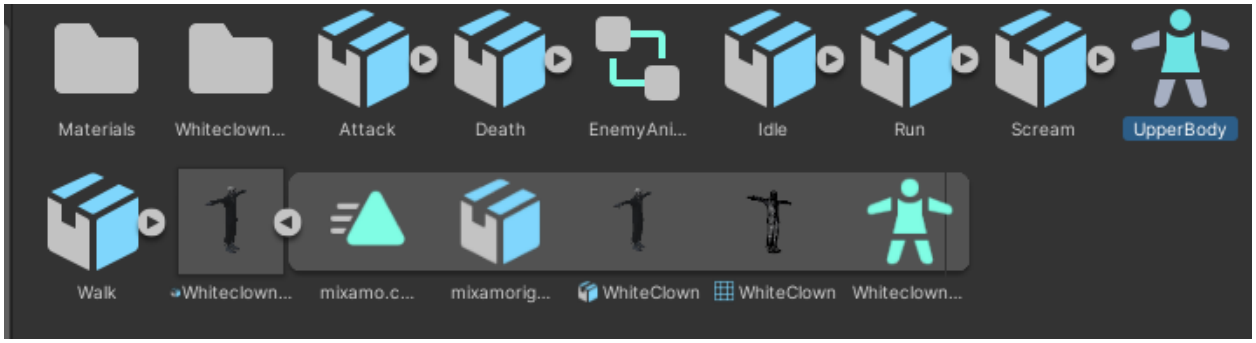
Now we will need to create an avatar Mask In Assets/Animation we will create the avatar mask



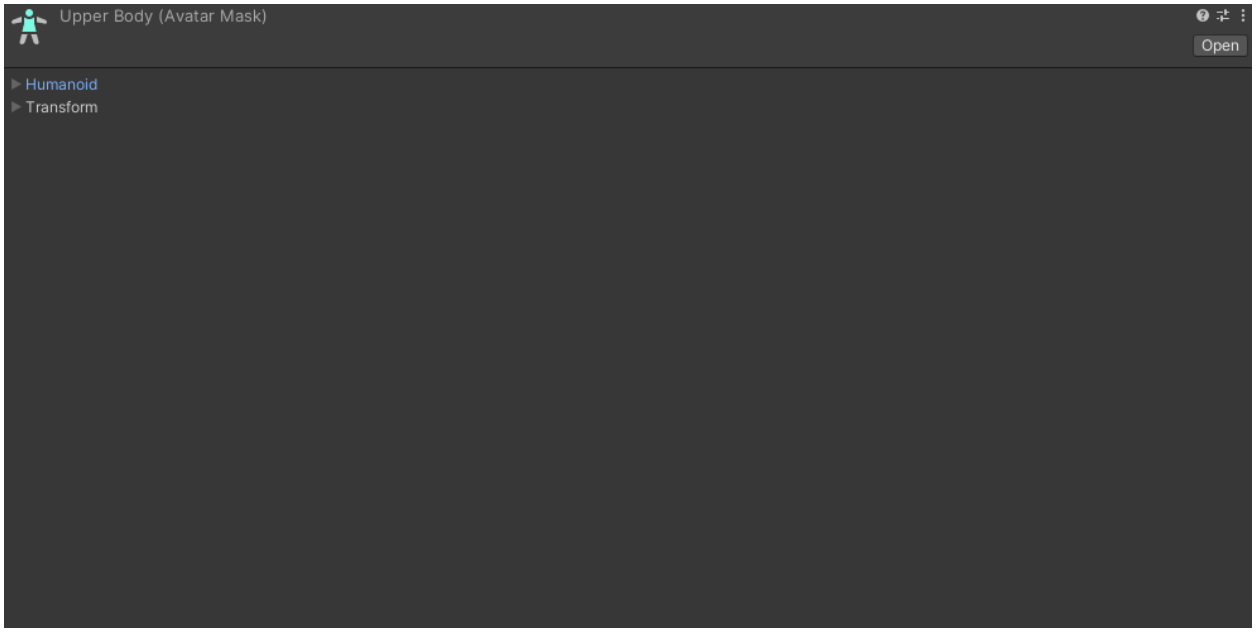
By left clicking and then choosing avatar mask in Create > Avatar mask



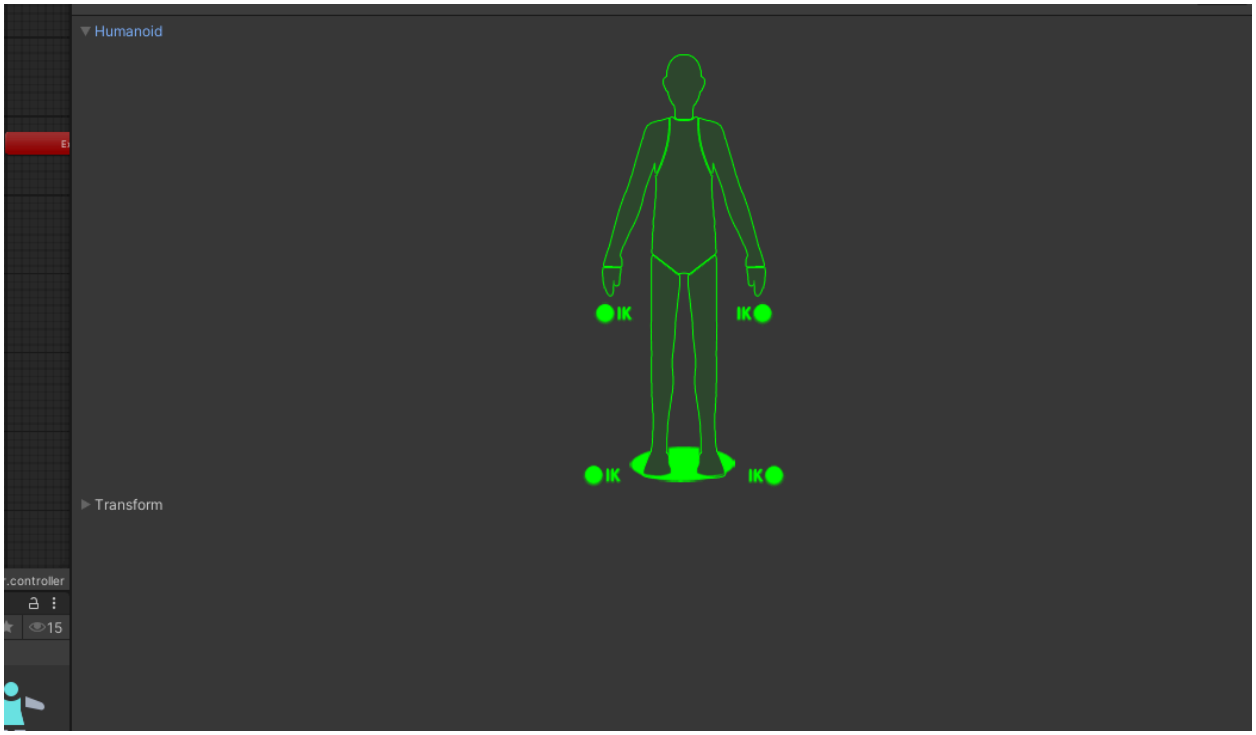
We will call the avatar mask Upper Body



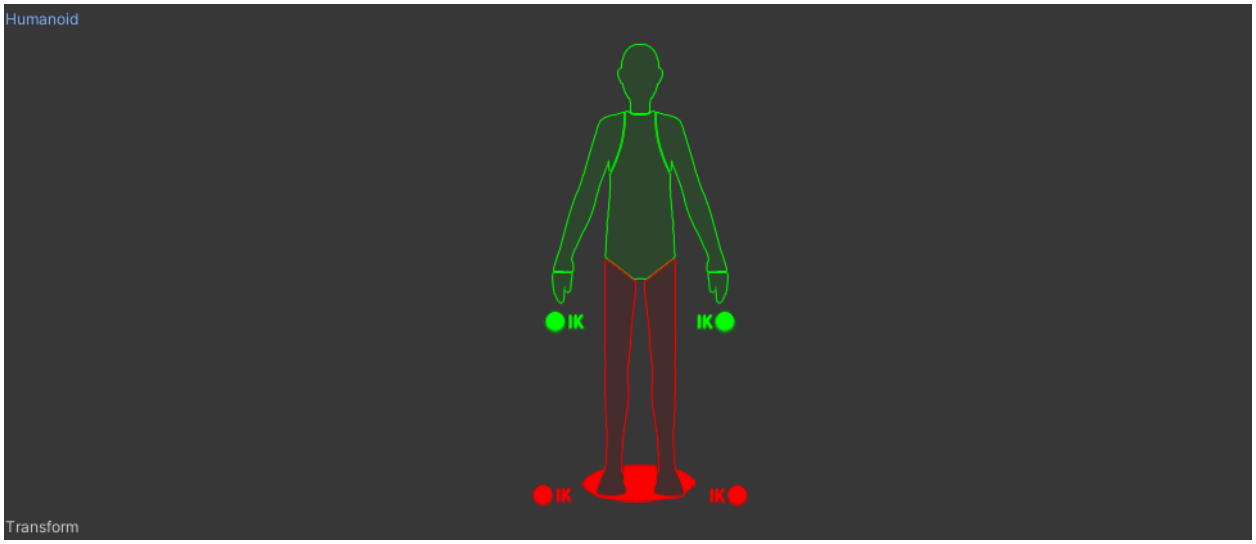
You should see this in the Inspector when you open it



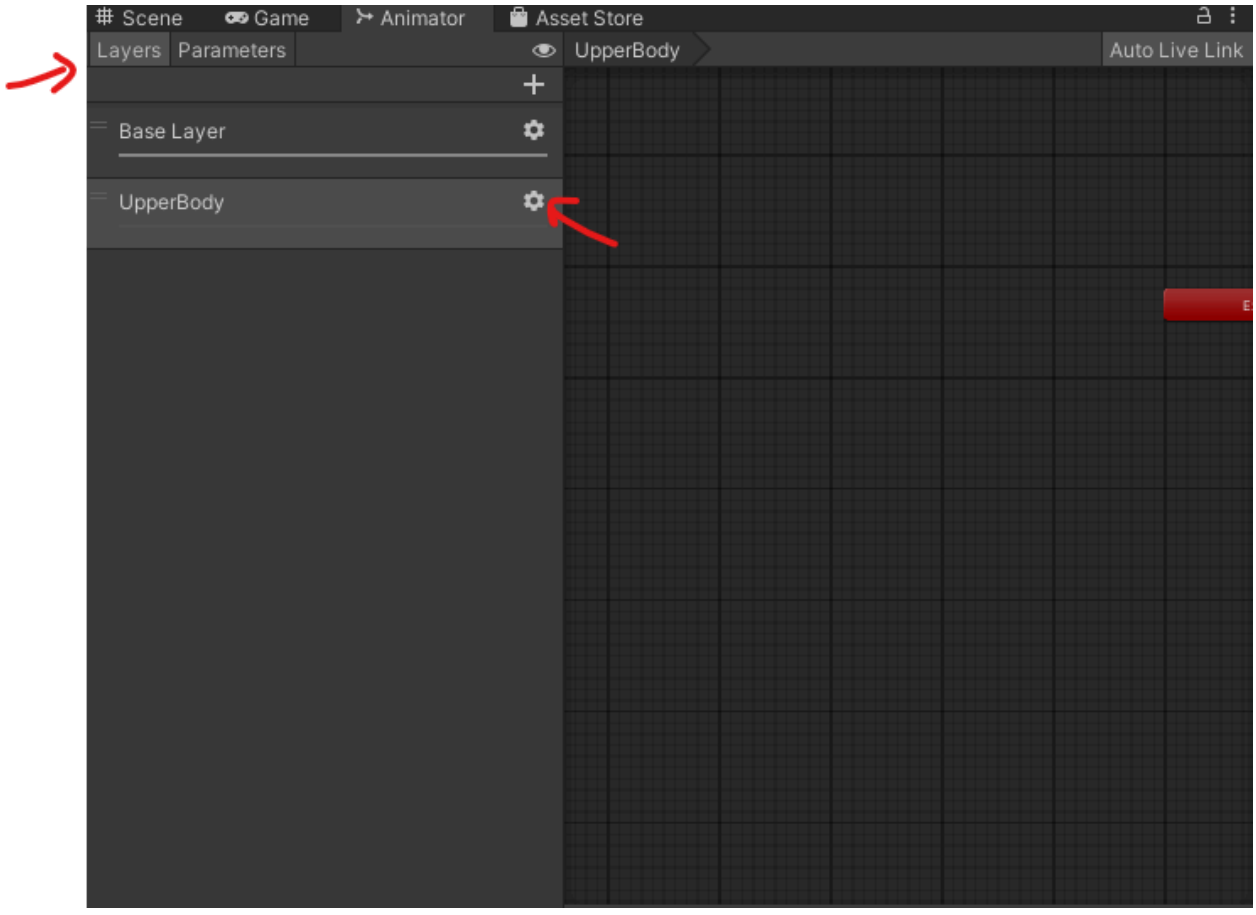
Open the Humanoid Tab



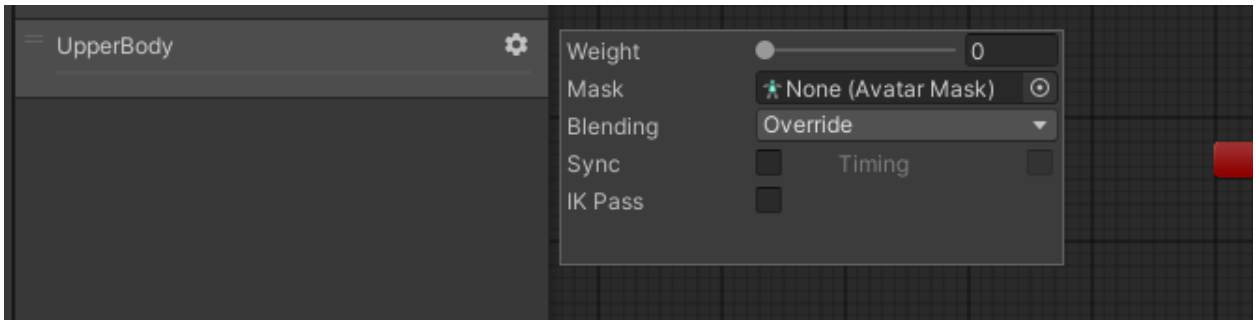
Click on the green areas to make them go red
We want to only have the upper body highlighted green like so :



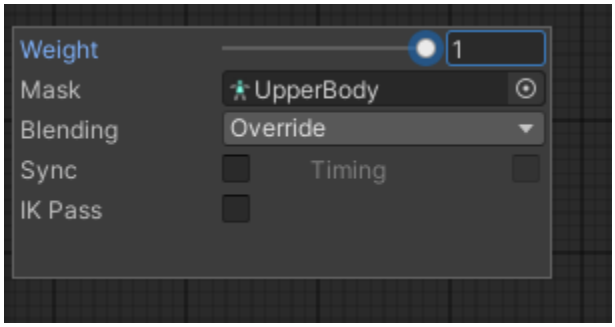
Now we will want to add that mask on our layer. In the Animator open the layers you should see a little gear icon



It will open this window. We want to add the mask in the mask section and put the **Weight** to 1.

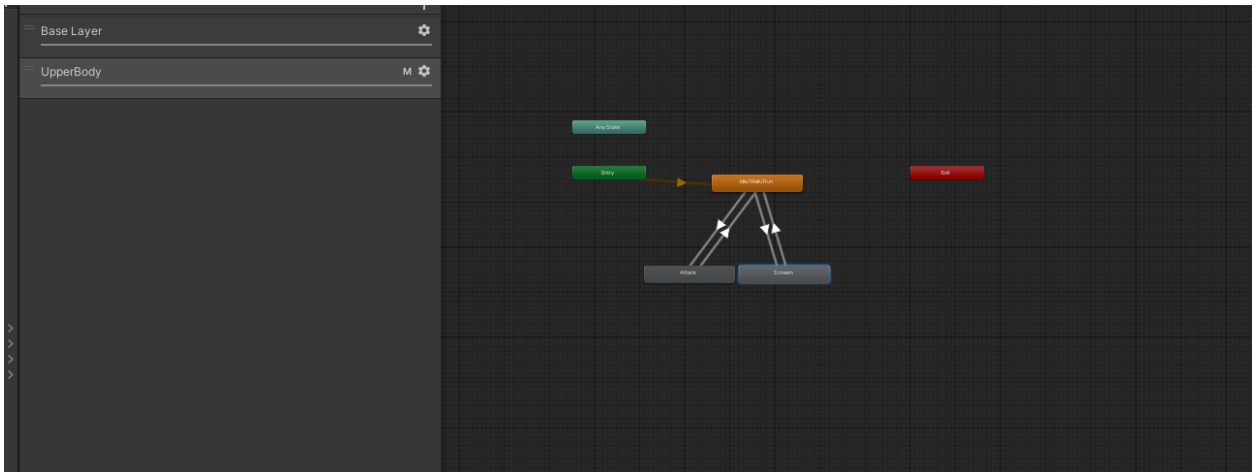


Like so

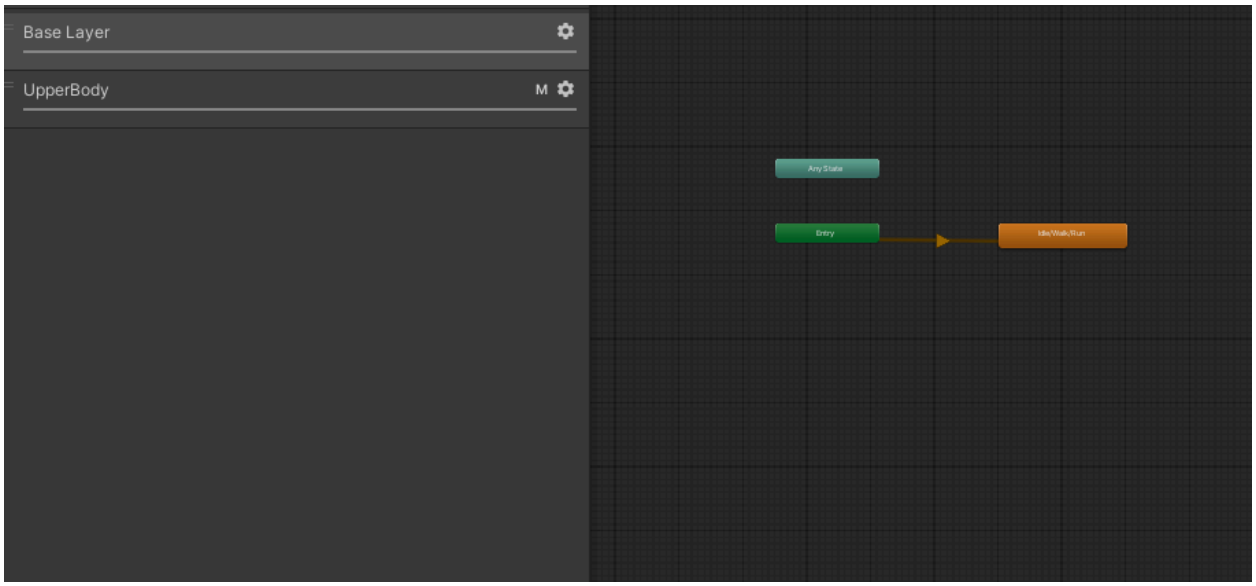


Now if you click on the Upperbody layer there should not be any animation

Copy and paste ones in your Base layer to your Upperbody layer



Then remove the scream and attack animation from your base layer



Your scream and attack animation should now only affect the upper body of your character.

You can now uncomment the scenemanager in AIAnimationController in HitPlayer

```
void HitPlayer() // Attack Animation Event to check if the player his hit
    {
        GameObject objectHit;
        if (aiController.CheckHandsCollision(out objectHit, "Player"))
        {

            SceneManager.LoadScene(0);
        }
    }
```

You should now reload the level when the AI hits you. (note : you have to be very close to him for the hit detection to work)